More details about the PAGI project

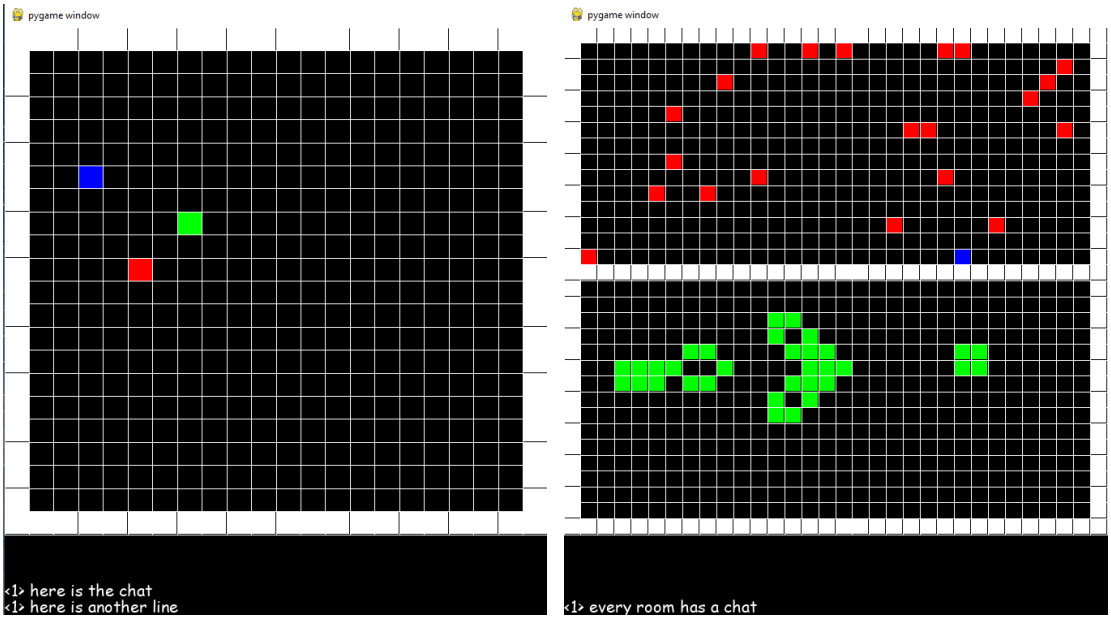
Didier Le Bail

January 14, 2026

Abstract

Description of the PAGI project. This description is incomplete, because I started it a few days ago to strengthen my job application. I am looking forward to giving more details at the occasion of the job interview, if needed.

Contents



(a) **Interactive screen.** Multiple agents can modify the inner pixels of the same interactive screen.

(b) **Dream room.** In the dream room, agents can watch various animations. Here, the top animation is custom and the bottom one is the game of life.

Figure 1: **Visualization of some of the world content.** Note that this PyGame interface is for human visualization only ; the agents receive directly one integer-valued matrix and one string at each time step. They are a part of respectively the screen and the chat content shown here.

1 Introduction

1.1 Goals

Our long-term goal is not limited to finding an algorithm, that emulates Artificial General Intelligence (AGI). In addition, this AGI should be able to communicate its intentions and thoughts, and to explain its line of reasoning to humans. For safety and ethical reasons, it should also show an interest in cooperation, as well as having consideration for life forms. For practical reasons, it should be able to perform both theoretical and experimental research, as well as engineering in our world.

Finding such an algorithm is usually considered as an optimization problem, and as such follows the general structure:

- defining a training objective
- defining a class of trained models
- defining a training method

The PAGI project proposes to explore an original approach for each of these aspects, by drawing mostly on the fields of artificial life and neuroevolution.

1.2 General setup

We consider a population of agents immersed in a virtual world of our making. This world is deterministic, Turing-complete and has a discrete clock. More details about it can be found in section 4. The world content, that is directly available to the agents, can be interpreted by a human as an image and a chat (see Figure 1 for two examples). What we call an agent here is a computing system (controller) interacting with the world through an interface. In addition,

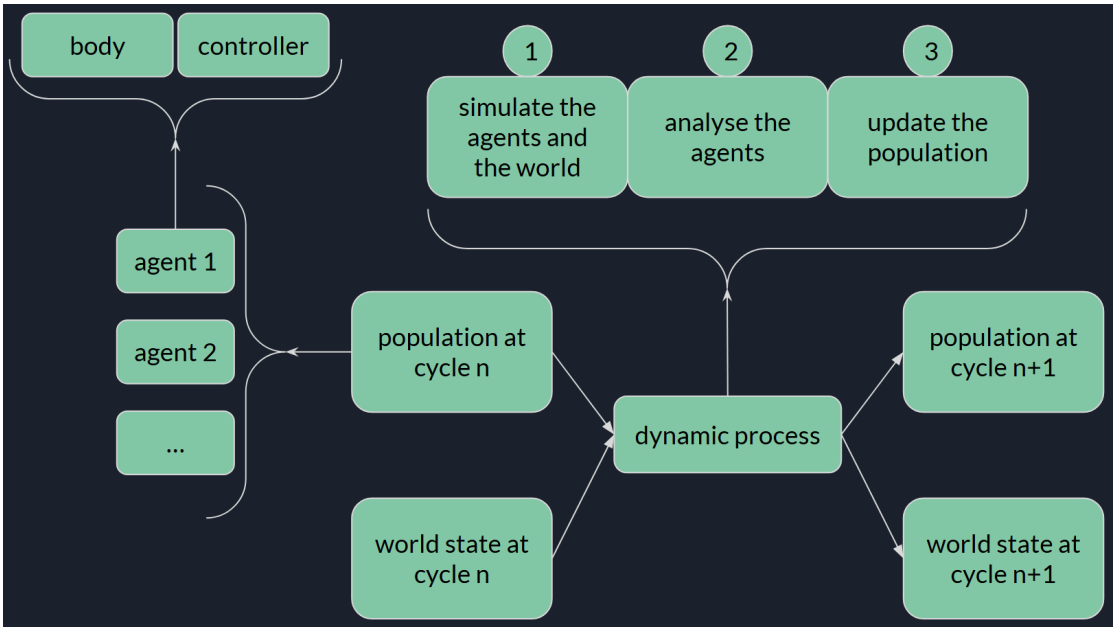


Figure 2: **Scheme of an evolutionary run**. Two types of arrow can be seen, arrows that indicate a flow of information from one box to another and arrows that indicate the content of a box. Once a population of agents and a world have been initialized, they are evolved along a run through cycles. The state at cycle n is obtained by applying iteratively a dynamic process n times to the initial state. Note that only the controllers are evolved, the bodies of the agents being kept the same.

an agent has a genome, from which its computing system is decoded (see section 3). Although many existing models could be considered for the controller, I preferred to build another one, described in section 5. The controller's interface is called a body by analogy with the biological case. Each agent is controlled by a separate system, and the agents are able to communicate with each other via a one dimensional channel (the chat pictured e.g. in Figure 1). At each time step, each agent receives one integer-valued matrix and one string. Importantly, these are of fixed sizes, while the world content may be of variable size (e.g. the chat increases in size as new messages are posted on it). This defines a vision and textual sensory fields, that the agent can move to access the whole content of the world. An agent's controller sends to its body a string and an action. This string is posted on the chat, to which the agent is connected, and the action allows the agent to either modify the world content or to access another location in the world. Actions are discrete and so both posts and sequences of actions can be modeled as streams of tokens (strings). This makes it possible to easily mine for patterns in those streams.

The population of agents is modified in the course of an evolutionary run. Only the agents' controllers are modified, the bodies being kept the same. These modifications are decided upon statistics sampled from the population trajectory, and made in agreement with a training policy defined before the run takes place. An evolutionary run consists in running repeatedly these three consecutive steps (see Figure 2):

1. simulate the agents until the sampled distributions have converged numerically
2. pause the simulation and analyze the collected data (in practice, they are stored in a SQL database)
3. update the population of agents according to the current training policy and resume the simulation of the world

1.3 Training objective: overview and motivation

Given our specific setup (virtual world and agents), what would be the observable consequences of AGI? How should our agents behave if they were generally intelligent, or had a subjective experience of their world? We refer to such behavioral properties as the prerequisites of AGI, as they should be shared by any population of AGI agents. We can then formalize such prerequisites as particular distributions for some observables sampled from the free behavior of these agents. Thus the training objective is, that every observable of interest should follow the identified target distribution.

Here, it is important to distinguish between properties (do not depend on our specific setup) and observables (depend on our specific setup). The definition of the training objective can be pictured as follows:

1. define a target property (for example, "the more an agent knows, the faster it learns"), that may be shared by every AGI agent
2. introduce, in the chosen setup, a finite set of observables together with a set of target distributions, such that:
3. the target property is satisfied in the setup iif every observable of the set follows its target distribution, when sampled from the free behavior of the studied agent

In section 6, we detail the eight properties we have identified so far and how we chose to probe them in our setup. An evolutionary run is said to be successful if at least one agent satisfies every target property.

2 The intrinsic limitation of the PAGI project, and training method

We use a human-guided evolutionary algorithm to update our population. This is a deliberate choice, in part because of the proven efficiency of human-computer collaboration, but also for deeper reasons.

Indeed, usual genetic algorithms fail to produce agents above some algorithmic complexity threshold, due to up to three reasons (depending on the precise algorithm):

- random mutations, that limit the size of a coherent genome below a finite length
- loss in diversity: all genomes in the population converge to the same value, which prohibits the algorithm to explore new genomes
- ***non-adaptive selection pressure***

The third reason is the deep one: Any selection pressure has its scope limited to a bounded range of algorithmic complexities. In practice, this means that beyond a level of algorithmic complexity, the selection pressure will not anymore be able to identify the fittest agent reliably. Since most pressures of selection used in practice are frozen, in the sense that their scope does not evolve with the population of agents, then the agents evolved are intrinsically limited in algorithmic complexity.

Of course, this would not be an obstacle if our optimization problem would admit a solution in the working range of our selection pressure. But, since AGI is likely to be found in a region of the search space with extreme complexity, this is a problem [1].

One workaround is to resort to human-computer collaboration, because humans have a larger working range than most automatic methods. However, this does not remove the limit com-

[1] Note that this complexity is relative to the chosen framework. If we had a better understanding of what AGI is, we could design a genetic encoding such that the AGI agents have simple genomes.

pletely. To do this, another project is needed [2]. In the PAGI project, we assume that human-guided evolution is powerful enough to already observe some interesting results.

In order to take informed decisions, the humans have access to a dashboard and are constrained by a training policy, of which I gave an example in my research interest statement ("A novel non-greedy optimization algorithm"). It is described in more details in section 7. The humans guides are further limited by a finite set of actions. At each cycle, they must choose a single action from this set, to update the population. Let us now give more details about these actions, and about the genetic encoding we use.

[2]Agents will be taught how to perform the selection by themselves. Over generations, the initially external selection pressure, of frozen range, is progressively substituted with a social selection pressure, whose range will adapt to the population. We get a virtuous cycle, since the more intelligent the agents are, the better they are able to select for more intelligent agents, and so on. Ask me directly for more details.

3 The genetic encoding and mutations

As mentioned in subsection 1.2, the controller of an agent's body is decoded from its genome. Since this controller is plastic, it will then evolve during the agent's life (see section 5).

A genome can be seen as an algorithm (sequence of instructions) that returns an artificial brain (the name of the neural model we use as a controller, see section 5). Many choices exist to parametrize such a space of algorithms, leading to different types of genomes:

- **an abstract syntactic tree (AST):** the nodes are primitives of a domain-specific language (DSL). Eventually, new primitives can be defined as the most redundant sub-trees (seen up to β-equivalence) ; a strategy already implemented in the literature (see DreamCoder).
- **a binary string:** a common encoding, often the binary representation of the model's trainable parameters.
- **a geometric encoding:** see hyperNEAT in the literature for more details.
- **a direct encoding:** this genome is equal to the phenotype, which here is an artificial brain. Since an artificial brain has a rich structure (see section 5), this encoding may yield good results if combined with NEAT (see the literature for details about NEAT).

What is the best choice? What properties should a good genetic encoding have?

3.1 Fractal parametrization

3.1.1 Theory (conjecture)

Let us formalize a bit our evolutionary setting. Let us introduce the space of genomes G, equipped with a metric d_G. This metric depends on the choice of the mutation operators: the distance from a genome g to another g' is the shortest number of mutation operators we need to apply to g before we get g'. Note that is the Kolmogorov complexity of the algorithm that rewrites g into g', when the underlying programming language is generated by the chosen mutation operators. For this reason, we say that d_G is a *Kolmogorov-complexity-sensitive metric*, or more simply a Kolmogorov-sensitive metric. For example, if G is the space of binary strings, and that the mutation operators are the bit flip, bit insertion and bit deletion, then d_G is the edit distance. Another common example is the usual distance between natural integers. Here, the mutation operators are "increment by 1" and "decrement by 1". However, note that this does not define a Turing-complete language. This can be seen in 2 ways: (1) Kolmogorov complexity invariance is not satisfied, since the complexity of the algorithm "increment by n" is not bounded with respect to n ; or (2) we cannot write e.g. the algorithm "multiply by 2".

Let us introduce a phenotype space, or evaluation space E, which here is the space of artificial brains. The goal is to find an artificial brain that satisfies some criterion (e.g. the prerequisites to AGI). Although we have no clue about how the set of solutions is distributed in E, we may assume the following (qualitative):

- the set of solutions is somewhere in a very high complexity region of E
- however, many of the solutions are computable

Algorithmic complexity comes into play because a genome is actually an algorithm returning an artificial brain. Thus, the decoding map $\phi : G \to E$, which maps a genome to the artificial brain it produces, takes only computable values (so of finite complexity).

In this setting, is there a choice of encoding (i.e. d_G) that allows us to find a solution faster than a random encoding, when the set of solutions satisfies the hypotheses above? [3]

Let us describe our conjecture. We introduce a metric d_E on E, which, just like d_G, is the Kolmogorov complexity of a rewriting algorithm evaluated in some language. Besides, we impose

[3] If the set of solutions does not satisfy particular hypotheses, then the no-free-lunch theorem tells us that all encodings are equivalent.

that this language is Turing-complete. See subsection 3.2 for an example of such a metric in the case of artificial brains. Then, we say that d_G defines a *fractal parametrization* of G, with respect to (E, d_E) and ϕ, if (qualitative):

$$\forall g \in G, \exists \alpha > 0, \forall \epsilon > 0, \begin{cases} \mathbb{E}_{g' \sim \mathcal{U}(\mathcal{B}_\epsilon(g))}[\phi(g')] = \phi(g) \\ \mathrm{Var}_{g' \sim \mathcal{U}(\mathcal{B}_\epsilon(g))}[\phi(g')] \sim \mathrm{diam}\left[\phi\left(\mathcal{B}_\epsilon(g)\right)\right]^\alpha \end{cases}$$

where $\mathcal{B}_\epsilon(g)$ stands for the open ball of radius ϵ centered on g and $\mathcal{U}(A)$ denotes the uniform measure on A. The variance is defined here as

$$\mathrm{Var}[\phi(g)] = \mathbb{E}\left[d_E\left(\phi(g), \mathbb{E}[\phi(g)]\right)^2\right]$$

and the diameter of a part of E is defined as in the literature:

$$\mathrm{diam}\left[A\right] = \sup_{x,y \in A} d_E(x, y)$$

Although this definition of a fractal parametrization is motivated by the study of power-laws, it should not be taken too seriously: its role is to motivate some future mathematical research. An alternative definition, more physical, would be that a random walk in G maps to a Lévy flight in E. Here, a random walk could be defined by starting from a genome g, to which we apply mutation operators picked at random consecutively.

Given one or the other definition, our conjecture states that, when following a random walk in G, we will get closer to the set of solutions faster when d_G defines a fractal parametrization. I actually learned a few days ago that a Lévy flight is indeed an optimal foraging strategy in some cases (see the literature), which comforted me in the existence of a similar result here. One important point to notice, is that we propose it works whatever the choice of d_E, as long as it a Kolmogorov-sensitive metric that derives from a Turing-complete language. This is motivated by two facts:

- the solutions are supposed to be very complex
- the difference in Kolmogorov complexity between the same algorithm expressed in two different languages (here the choice of language is encoded in d_E), is bounded as soon as these languages are both Turing-complete. This is the so-called invariance of the Kolmogorov complexity (see the literature). Thus, the choice of Turing-complete language becomes irrelevant when we consider objects of a complexity that approaches ∞.

3.1.2 A practical example of a fractal parametrization

One possibility would be to start from d_E as described e.g. in subsection 3.2, and to consider a binary string encoding. Then, taking d_G as the edit distance, all what we have left to do is specifying the decoding map ϕ. The idea of this mapping is pictured on Figure 3:

1. Hierarchical clustering is applied on E by using d_E. This allows to represent the space of artificial brains as a binary prefix tree.
2. each path from the root to a leaf is seen as a binary string, used to encode the leaf.

We may call the obtained parametrization the *canonical* fractal parametrization associated to d_E, and study its properties in future work.

3.2 An example of Kolmogorov-sensitive metric between two artificial brains

Note that we assumed implicitly that E was a vector space, and that it is not obvious, how to embed an artificial brain into a vector space. However, note that in our recursive novelty search (see subsection 7.1), we give more weight to the external behavior. Then, we shall choose an embedding such that different brains means different input/output mappings. This may be realized by the following strategy:

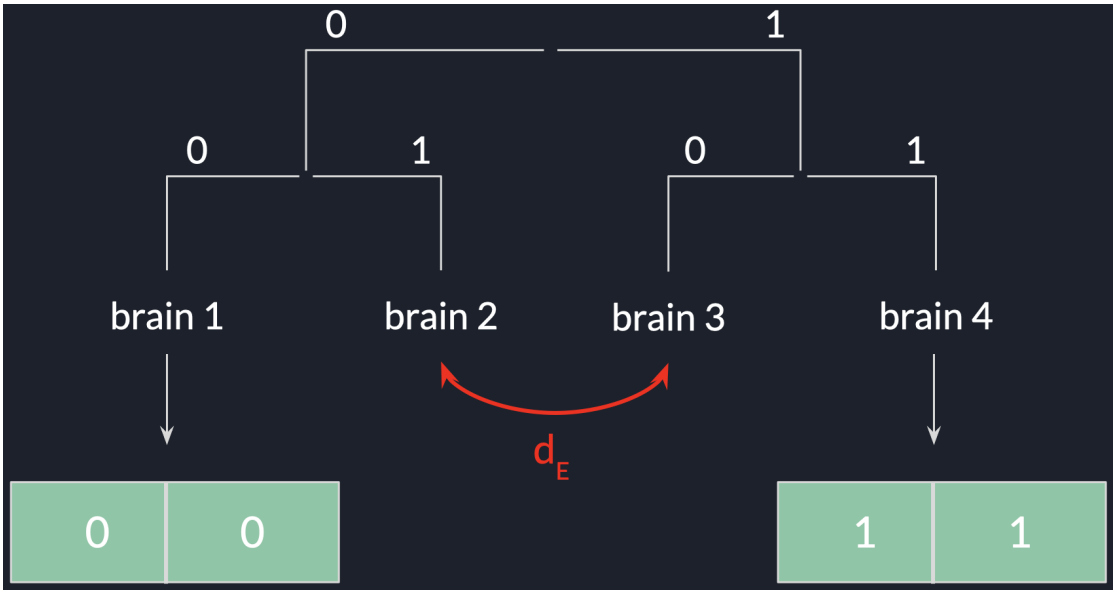


Figure 3: **Canonical fractal parametrization.** Given a (E, d_E), we can build an encoding map ϕ so that the space of binary strings equipped with the edit distance defines a fractal parametrization with respect to d_E. This is done by first partitioning E by hierarchical clustering. By mapping binary strings to paths in the resulting tree, we decode a string as the brain at the end of the corresponding path.

1. define a set of test input time series of sensory inputs
2. record the brain output separately for each test time series (note that the brain should start from the same state before being fed an input): this set of output time series defines the *brain signature*

Since the space of brain signatures is a vector space, all we have left to do, is to equip it with a Kolmogorov-sensitive metric. This can be built out of common operations on time series, like translations, homotheties, time-reversal, etc. As said earlier, we believe that the precise choice of these operations matters less than the fact that they define a Turin-complete language. See Figure 4 for an illustration of the difference between the Euclidean metric and a Kolmogorov-sensitive metric in a particular example.

Actually, in the context of human-computer collaboration (see Risi *et al.*), I noticed that humans tend to compare behaviors by using a Kolmogorov-sensitive metric instead of a Euclidean

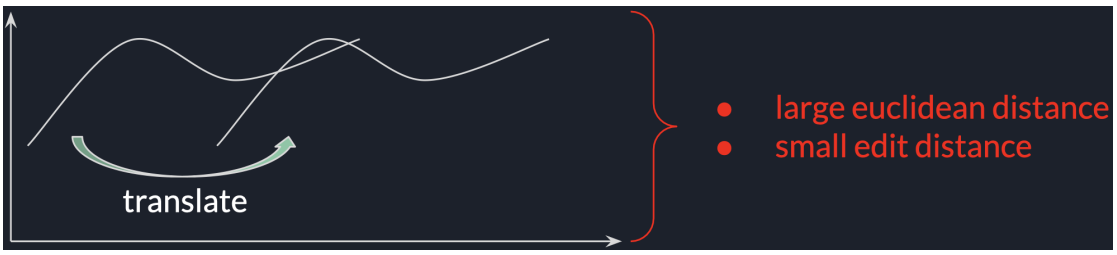


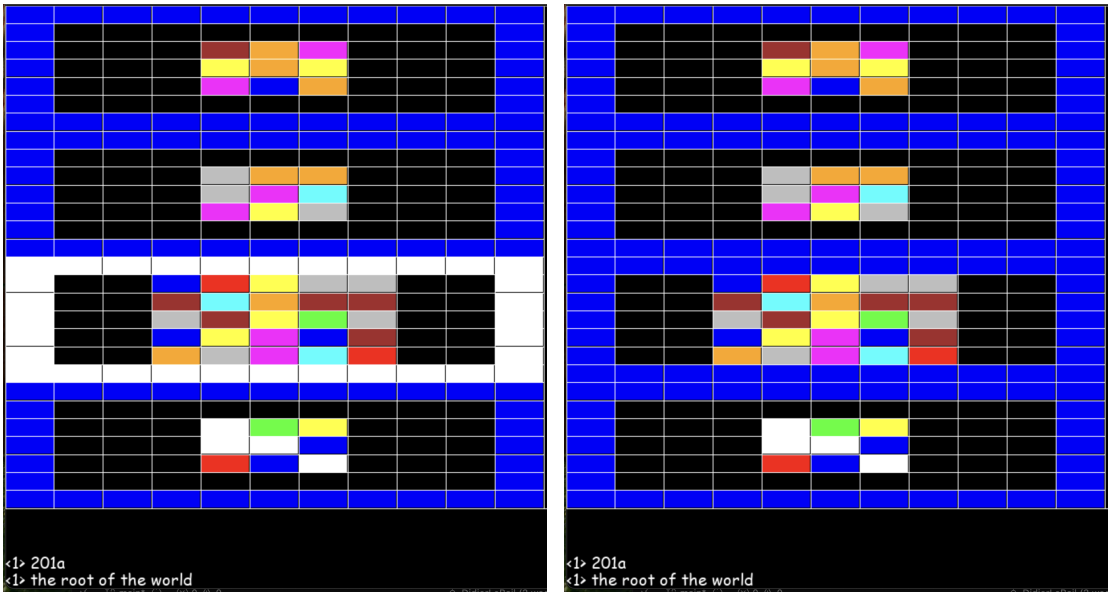
Figure 4: **Dummy example for comparing Euclidean and Kolmogorov-sensitive metrics.** On this example, two time series are shown, one being the time translated of the other. If we think of these time series as brain signatures, i.e. the brain response to the same sensory input, then we would conclude that these two brains lead to similar behaviors. If we use an Euclidean metric, we get to the opposite conclusion because the Euclidean distance between the two time series is large. If we include the time translation among the mutation operators, we reconcile with human insight.

measure. For instance, at some point in the experiment described by Risi *et al.*, a human guide considers that the two behaviors: "having a perfect score" and "doing the exact opposite of what should be done" are very close to each other. The human guide then selects for the agent with the *lowest fitness*, resulting in the ideal agent a few generations later. This is a motivation for considering a Kolmogorov-sensitive metric: if a simple symmetry were part of the mutation operators, then the two aforementioned behaviors would be evaluated as very close to each other, just like the human guide estimated them to be.

3.3 The mutations chosen in practice

This part of the project is still work in progress, but we have the following ideas. First, an archive of agents should be maintained, which is usual practice for both Darwin-Gödel machines and novelty search. Then, if we consider the genome to take the form of an abstract syntactic tree, we want to include the following possibilities for the human guides:

- protect / target some regions of the genome (similar to the innovation protection mechanism of the literature, but controlled explicitly here)
- various unary operators: subtree refactoring, pruning, insertion, swapping, duplication, etc.
- binary and higher order operators: we define a n-order crossover as an operation taking n genomes as input and returning n genomes as output. Each output genome is obtained by combining fragments from the n inputs.
- choose the agents either from the current population or the archive
- place an agent in the archive. For example, place each agent that beats all previous agents at some metric: the new closest agent from the training objective ; the new most novel agent, the new simplest agent, etc.



(a) **Cursor visible.** (b) **Cursor invisible.**

Figure 5: **Root of the world.** The world root has itself as parent location. Its children locations can be accessed via a visual menu. Here, the root has 4 children, each child being associated a different logo (matrix). An agent select a child by moving a cursor, whose white border oscillates between a visible state and a transparent state. Pressing the "enter" button will move the agent to the location pointed at by its cursor. Note that here, an agent does not see or act on the cursors of the other agents. (a) the cursor is visible ; (b) the cursor is transparent

4 The virtual world

4.1 General structure: rooms and chats

The world that contains the agents has a static tree structure, where nodes are locations. The content of a location can be accessed via an integer-valued matrix and a chat. The matrix can be interpreted as visual data and the chat as text or auditory data. A given chat might be shared by multiple locations, and multiple chats might be available from a given location. Generally, there are one public chat and one private chat, the latter being used to interact with some component present at the location. It is also used to signal some events to the agents, like "the agent ¡id¿ entered", or "the agent ¡id¿ left", and so on.

Besides, each location comes with a list of buttons, which when pressed by an agent may have some effect. For example, every location has at least 1 button, called the "return" button. When an agent activates this button, it moves to the parent location of its current location. Locations with multiple chats have another button, that allows an agent to switch between the available chats.

For the non-leaf locations, at least 2 other buttons are available: one to select a child location and one to enter it. On Figure 5 is an example of how an agent can visualize and select the "doors" leading to each child location.

The particular world we consider is divided in 4 rooms, that can be entered from the world root (see Figure 5). A room is a particular subtree of the world, and thus might contain several locations. Let us describe these rooms, by giving more illustrations than details.

4.2 The rest room

This room has a single location, characterized by an empty (black) visual content. I designed it, to see whether agents would show a different brain activity in the absence of sensory stimuli,

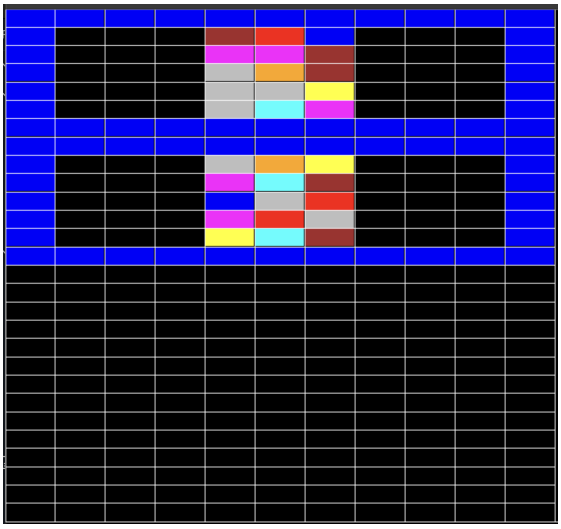


Figure 6: **Root of the blackboard room.** Here is an illustration of the visual menu (list of doors) shown to the agent after entering the blackboard room. The upper (resp. bottom) door leads to the pixel (resp. operator) wise setting. In any case, the visual display of the new location will be the same public screen. The public chats are also the same, but the private chats of the pixel and operator wise settings differ from each other.

and in particular whether they would use this room to sleep (sleeping is one of the most ancient behaviors of neural organisms).

4.3 The dream room

This room has for now a single location, but I plan to add other ones, to have more animations available. In the dream room, agents can watch animations, that play in a never-ending loop. These animations cannot be paused or affected by the agents. The Figure 1 (b) showed a frame of the 2 animations I have implemented for now.

4.4 The blackboard room

This room has 4 locations. Its central component is an interactive screen, or blackboard. As the name suggests, the agents can modify the content of the a blackboard. This blackboard is public, meaning that it can be modified by any agent connected to it. The agents can know who is connected to the blackboard by e.g. consulting the chat. An agent can modify a blackboard in two ways: either pixel wise, or operator wise (see Figure 6 to see how an agent can pick either way).

In the pixel wise setting, an oscillating cursor is associated to the agent. This cursor is visible by every agent connected to the same blackboard, and points both at location on the screen and a color. The pointed color and location are updated by pressing buttons.

In the operator wise setting, an agent interacts with a second component, called a library. A library can be thought as a database of operators, which take as input an integer-valued matrix and return an integer-valued matrix with the same size (see Figure 7 for an illustration). Each operator comes with a definition and a description. Its definition can be thought of as the code source of the operator, while its description is supposed to play the role of a documentation for the operator.

Agents can select and apply an operator to the screen content, but also read and modify its description. In the future, we will allow the agents to also modify its definition, and to introduce new operators into the library database.

The library database has a tree structure similar to a room, but navigation is done through a private chat. This allows the navigation in the library tree to be enriched with search-and-find

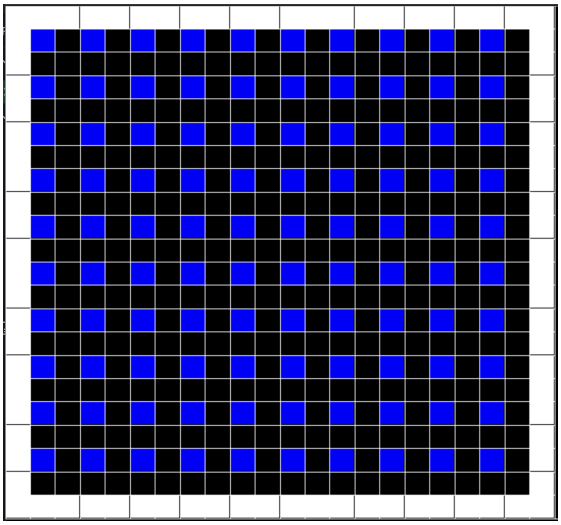


Figure 7: **Illustration of the tiling operator.** An agent can modify multiple pixels at once by applying operators, selected from a library. Here we started from a black screen, and applied the "tile" operator. For now, the other available are "change_color", "transpose", "rotate" and "translate". Note that we have not displayed the chat, because it is irrelevant here.

tools, similar to the ones available in any modern text editor.

Let us succinctly describe how we navigate within the library tree. At each node of the tree, the currently selected child is indicated as a message in the private chat. Entering this child location or going back to the parent location is done by pressing the same buttons as in a room. The only difference is that here, the updated content is the private chat instead of the visual display. Selecting another child can be done either by scrolling or by search-and-find: in this case, the message posted by the agent on the private chat is interpreted as a search request, and the children will be sorted by decreasing distance to the request, similarly to a simple search engine.

Let us now describe the levels of the tree. At the root, the children are the operators contained in the database. The selected operator is indicated to the agent by a name, that is posted on the private chat. After entering an operator, the children become: the operator description, the operator definition and the possibility to apply the operator to the screen content. If the agent enters the operator description, the private chat turns into a text editor: the agent sees a part of a text file, which it can scroll down or up. It can write to this file, and search-and-find strings in it. We will not detail here how we have implemented all this, see Figure 8 for an illustration.

4.5 The exam room

The exam room (see Figure 9) is the most complex one. In a nutshell, it allows the agents to access to the tasks of the (excellent) ARC-AGI challenge. The precise data we use can be found here.

When a task is solved, it is indicated on the public chat associated to the whole room, which agents have contributed to the solution. Importantly, agents can collaborate to solve the same task, and propose a given task to a given agent. This might allow (1) an agent to request help from a particular agent to solve a task, and (2) simply a task to be sent to the agent most likely to solve it.

However, note that we are NOT trying to solve the ARC-AGI challenge ; we are only using it to introduce a collaborative setting for our agents. Then, we analyze how they behave in this setting, but will not select specifically the agents that solve the underlying tasks (if there is any). Instead, we will follow the training policy 2, described in 7.

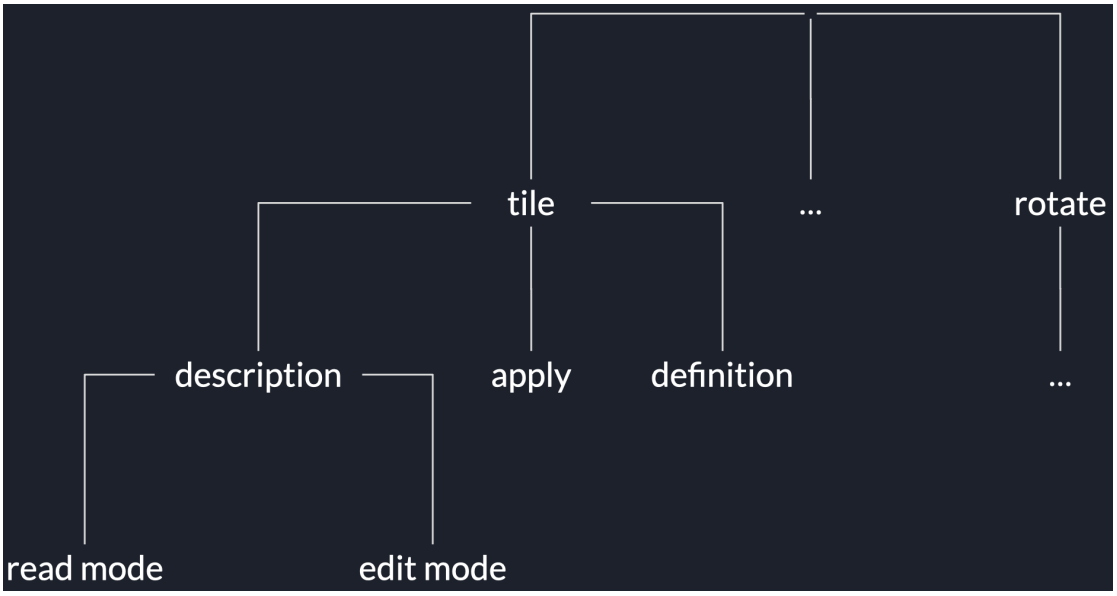


Figure 8: **The library tree.** The nodes "..." correspond to the unexpanded subtrees. Navigation within a database of operators is similar to the spatial navigation in a room, but done through a private chat instead of a visual content. Each level of the tree corresponds to a location of this chat environment. When navigating, an agent is located at a single node of the tree. It moves vertically by entering or quitting locations, and horizontally by pressing the other buttons available at the location. At the root (level 0), the agent has access to a list of operator names. It can scroll down or up the list, or use a search engine to find an operator faster. At level 1, the doors available are the operator execution ("apply"), description and definition. At level 2, the agent has access to a text editor when entering the description of an operator. If it applies it, the visual content is modified accordingly.

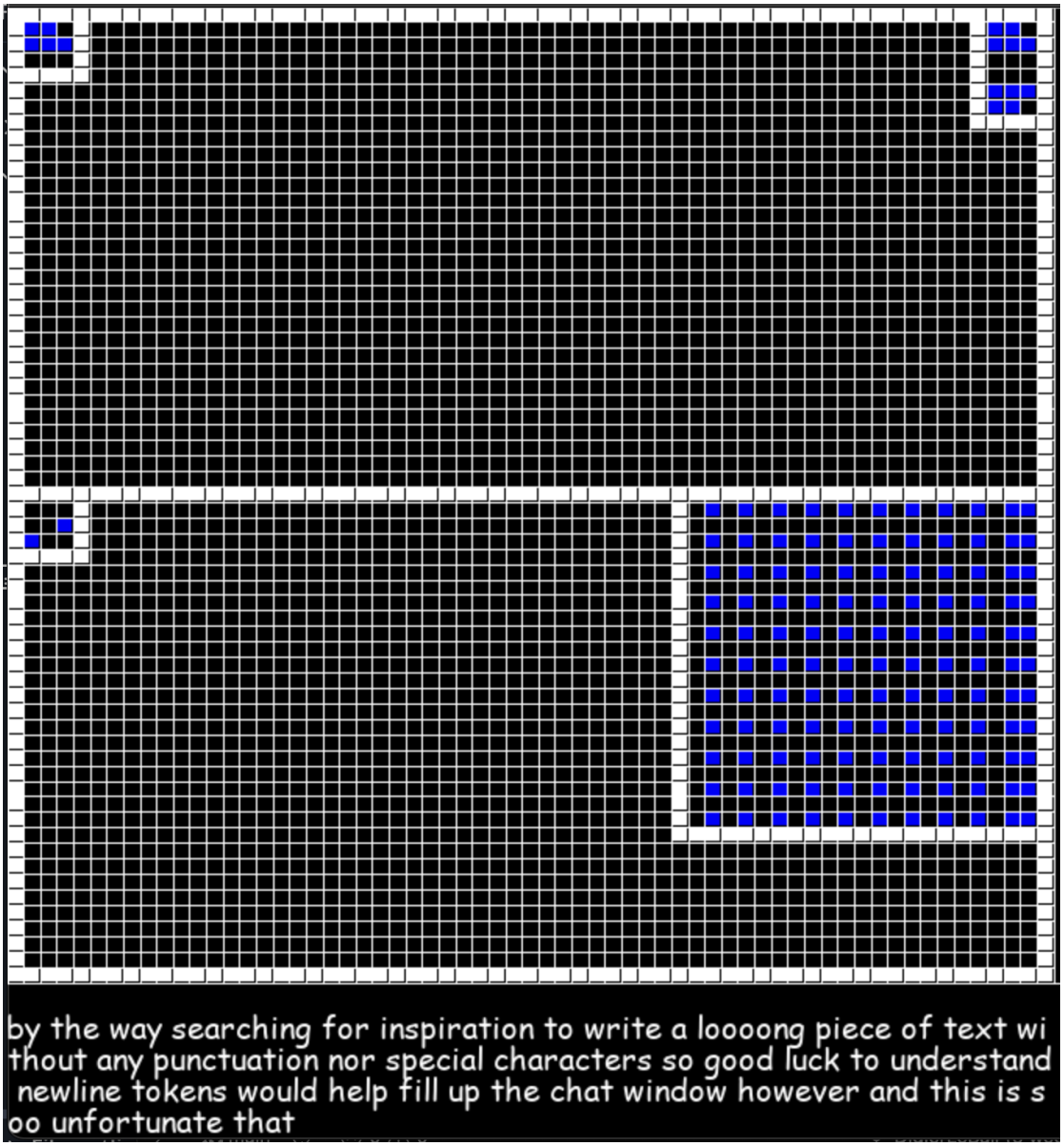


Figure 9: **Quick look at the exam room.** A ARC task consists in guessing an input/output mapping between integer-valued matrices, from a small number of example input/output pairs. Here these pairs are visible in the top part of the visual content. A single pair is shown at once, but the agent can scroll to access the others. In the bottom left part is shown the test input. In the bottom right part is an interactive screen, containing the test output proposed by the agents. It is the same type of screen as the screen of the blackboard room, except that the agents can here modify its size. When the content of this screen matches the ground truth for the test output, an indicative message is posted on the chat (given our training objectives, it is more aesthetic than useful). On the shown illustration, the test output has been obtained by applying "tile" and then "translate". The private chat is here connected to the (dummy) description of an operator.

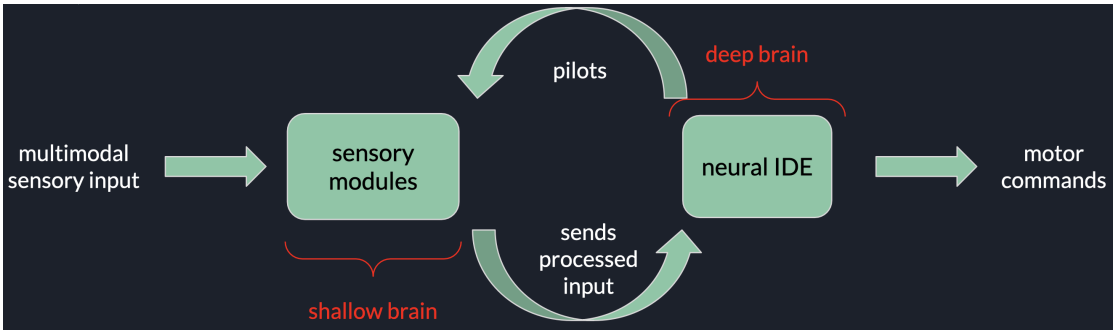


Figure 10: **High-level structure of an artificial brain.** An artificial brain can be seen as a deep brain (neural IDE) piloting itself and a shallow brain (sensory modules). The shallow brain is used to transform the sensory data into a meaningful representation, while the deep brain performs intricate computations. These include modifying its own architecture in a non-greedy way or modulating the computations performed by the sensory modules.

Figure 11: **Conventions when drawing processing units**. The input to a component is drawn as a rectangle tilted towards right. The output of a component is drawn as a rectangle titled towards left. The intermediate blocks of a component are drawn as straight rectangles. An arrow indicates how information flows from one block to the other at processing time.

5 The artificial brain

We call an artificial brain the model we use as a controller of an agent's body. At each world time step, this brain receives one integer-valued matrix and one string, both of fixed sizes. It also has a pool of computational resources, replenished at the beginning of each world time step. The brain draws from this pool at each elementary operation it performs, and may exhaust it. If this is the case, the brain is considered to have returned an output: the output layer of the brain is read and the corresponding motor command is sent to the agent's body.

An artificial brain has its own clock, and may update itself for a variable number of time steps, at fixed sensory input. This number is variable because an artificial brain updates itself until it returns an output. In the case it does not exhaust its pool of resources, the output is considered to be returned in two cases: (1) a "completed" signal is raised, through the activation of a specific neuron ; (2) the brain has reached a stationary state, and thus keeping updating it consumes resources for nothing.

Structurally, an artificial brain is made of two main parts (see Figure 10): a shallow brain and a deep brain. The architecture of the shallow brain is handcrafted, and static at run time. However, the computations it performs depend on the feedback signals it receives from the deep brain. The architecture of the latter is self-evolving, and transmission of information in the deep brain is not limited to synaptic signals (see subsection 5.3).

Before we describe the shallow brain in more details, please have a look to Figure 11, to know about the conventions we will follow in our diagrams. Importantly, we never draw the piloting ports, unless explicitly specified. The piloting ports refer to the feedback connections from the deep to the shallow brain.

5.1 The shallow brain

The shallow brain has two sensory channels, one for processing text and one for processing integer-valued matrices. Since these two channels are similarly structured, we will describe here this general structure, sometimes illustrating with text, and sometimes with matrices.

The general structure of a sensory channel is depicted on Figure 12.

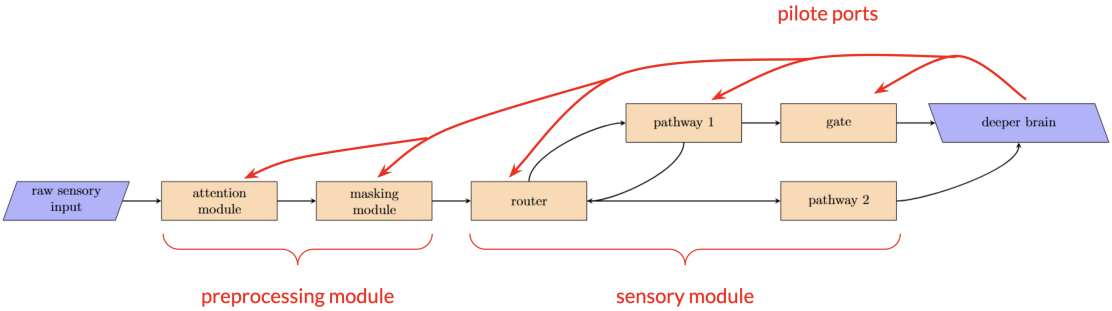


Figure 12: **General structure of a sensory channel.** The preprocessing module queries the agent's environment to extract chunks of fixed size. The sensory module turns these chunks of data into a representation, that can be processed by the deep brain. Note that 2 pathways are shown here, but there can be as many as desired. The red arrows represent the feedback connections from the deep brain to precise components of the shallow brain. See the main text for more details.

5.1.1 The attention module

This module returns the position, in the environment's coordinates, of a so-called attention window. Note that attention here has nothing to do with attention in transformers, but is more similar to their context window. This position is updated depending on the current position and the signals received from the deep brain: the latter can both *move* the window by a given amount or *place* it at specific coordinates. To indicate to an agent the boundaries of its environment, we allow the window to exceed these boundaries by one pixel. These pixels are filled with a so-called padding token (see Figure 13 for an illustration). Once the position of the attention window has been computed, the attention module sends a query to the agent's environment, in order to fill the window with the correct sensory data.

5.1.2 The masking module

The masking module returns both a position and a size. The position is expressed in the attention window's coordinates, and the size has the same shape as the attention window. Also, it cannot excess the size of the latter. Thus, the masking module designates a subpart of the attention window, whose all pixels will be assigned the value of a masking token (see Figure 14 for an illustration). If the masked area exceeds the boundaries of the window, it is wrapped with periodic boundary conditions. Thus, the number of masked tokens is only a function of the size of the masked area.

Both the position and size of the masked area are updated by the deep brain. The resulting data has the same size as the attention window, and are forwarded to the sensory module.

5.1.3 The sensory module

As visible on Figure 12, the sensory module receives its input from the masking module and forwards its output to the deep brain. Formally, its goal is to give to the brain access to an associative magma of transformations. This magma is generated by the families of functions, that each pathway implements. Said otherwise, a sensory module is able to compose various transformations of its input, and decides at which point to forward the resulting output to the deep brain.

To formalize this, let us take the example of a sensory module with the 2 pathways as depicted on Figure 12. Each of these pathways, being piloted by the deep brain, implements a family of transformations. For example, let us consider that the pathway 1 implements the family $(f_\lambda)_{\lambda\in\mathbb{R}^m}$, and that the pathway 2 implements $(g_\mu)_{\mu\in\mathbb{R}^n}$. Here, the input and output spaces of the pathway 1 are the same, which is why it can send its output back to the router. This is also why the pathway 1 is followed by a gate: the deep brain has to decide when to let the pathway output through. As long as the gate is closed, the pathway 1 can only send its output back to

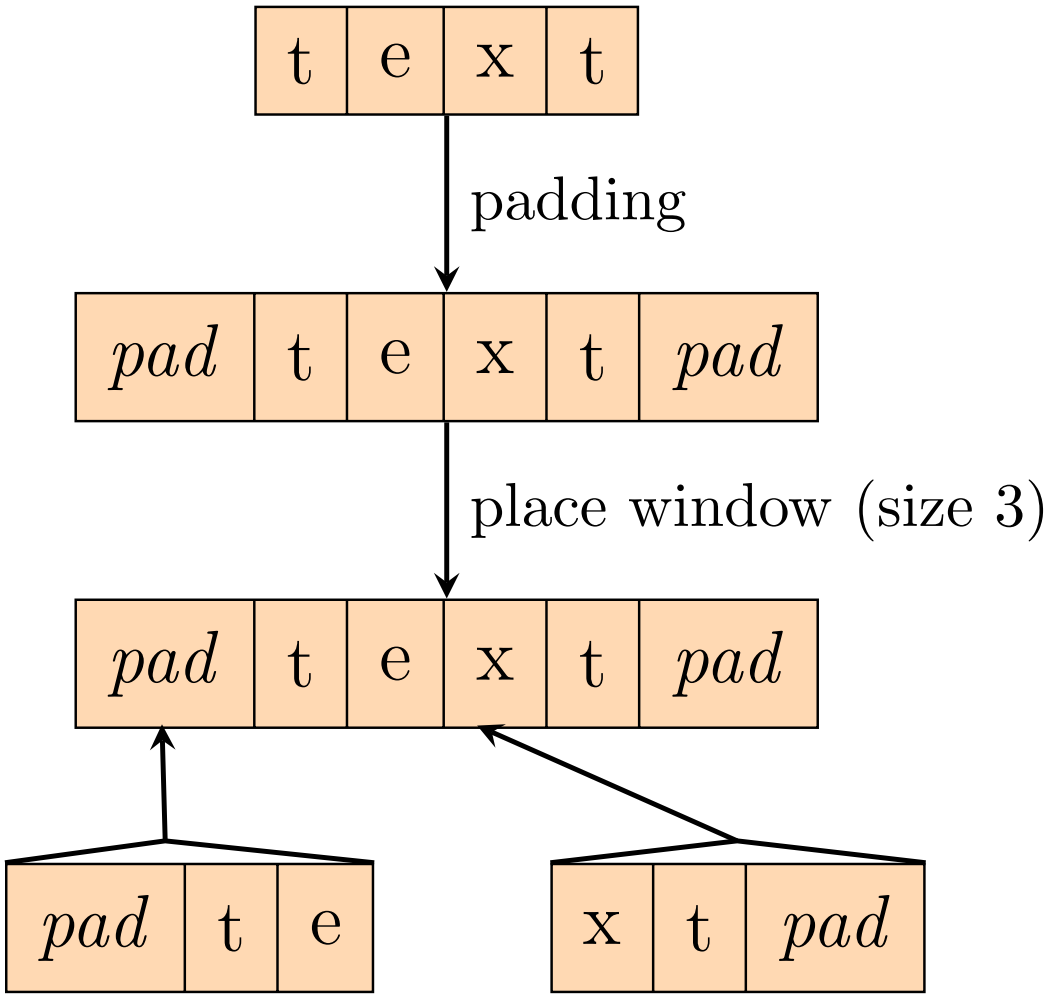


Figure 13: **Illustration of the attention module**. Here the agent's environment is a string of 4 tokens (chat content). The boundaries of this environment are extended with padding tokens, so that the agent can notice them. Once the window position is computed, it is filled with data from the padded environment. At the very bottom of the figure, 2 examples of position are given. For each position, the window content is displayed.

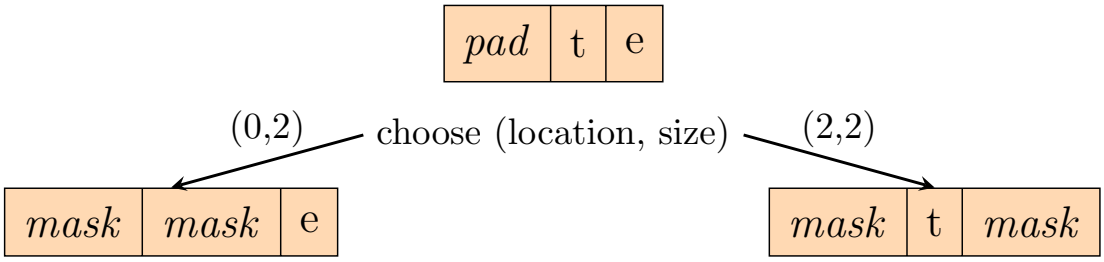


Figure 14: **Illustration of the masking module**. At the top is the content of the attention window. Here, we consider a masked area of size 2. Depending on its location within the window, it results in a connected or not connected masked area, due to periodic boundary conditions.

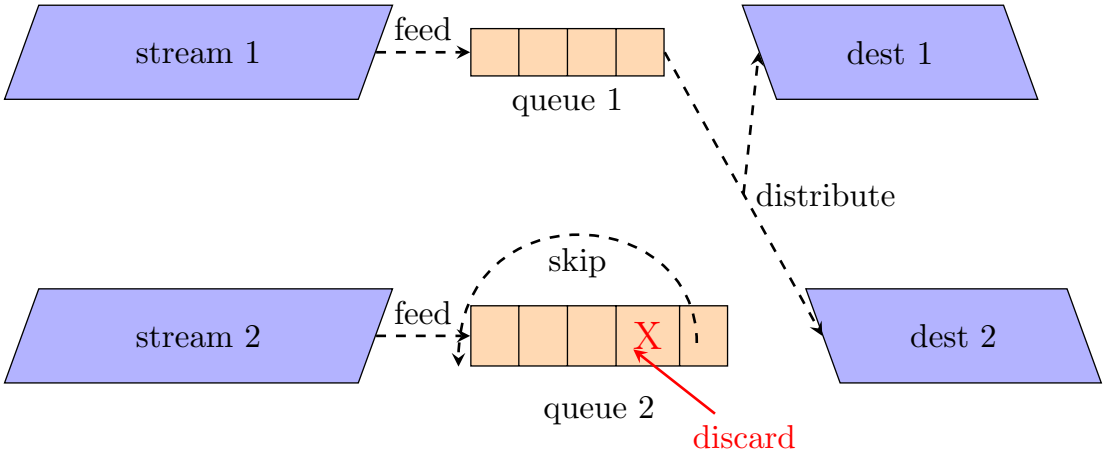


Figure 15: **Structure of the router**. The router distributes data accumulated in queues to destinations. While here 2 input and 2 destination streams are shown, a router can handle an arbitrary (bounded) number of them. We illustrated here the main operations that a router can perform: storing data in a queue (feed), moving it from the top to the bottom of a queue (skip), removing it (discard) and distributing it to destinations (distribute). A destination receives data from at most one queue. All operations performed by a router are decided by the deep brain through piloting ports.

the router. Then, the sensory module as a whole implements the following transformations of its input:

$$\{f_\lambda, g_\mu, f_\lambda \circ f_{\lambda'}, g_\mu \circ f_\lambda, g_\mu \circ f_\lambda \circ f_{\lambda'}, \cdots | (\lambda, \mu, \lambda', \cdots) \in \mathbb{R}^m \times \mathbb{R}^n \times \mathbb{R}^m \times \cdots \}$$

Where the choices of $\lambda, \mu, \ldots$ are controlled by the deep brain through the piloting ports visible on Figure 12.

Let us now give more detail about the various components of the sensory module, that allow it to implement the family of transformations we have just described.

5.1.4 The router (from the sensory module)

It chooses which data to send to which pathway. If no data are sent to a pathway, it is skipped, which can save computational resources. The structure of a router is visible on Figure 15. A router handles streams of data. Each input stream is associated a queue, in which the router accumulates the data it receives: At each tick (brain clock), the router adds the piece of data received in each stream to their corresponding queues. Second, it checks whether it should skip (data moved to the bottom of the queue) or discard (delete) the piece of data at the top of the queue. Third, it redirects the data at the top of each queue towards one or several destinations. This distribution satisfies one constraint: At each tick, a destination receives data from at most one queue.

All the choices made by the router (how to distribute data among destinations, whether to skip or discard, etc.) are imposed by the deep brain through piloting ports.

5.1.5 The pathways

On Figure 12, 2 pathways are shown, one that can compose with itself and one that cannot. In practice, we have implemented four:

- a symbolic pathway: its body performs symbolic transformations of its input
- a machine learning pathway: when handling matrices, its body is a convolutional neural network or a transformer (see Figure 17 for an example)

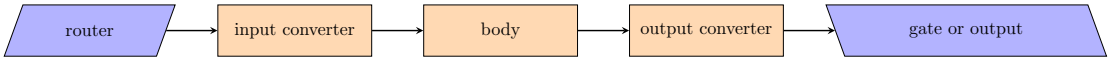


Figure 16: **Backbone of a pathway**. Only the body of a pathway is piloted by the deep brain. The input and output converters constitute the interface between the body and the other components. If the router sends no data to a pathway, it is skipped at processing time. If the transformations implemented by the pathway can be composed with one another, it sends its output to a gate (see Figure 12).

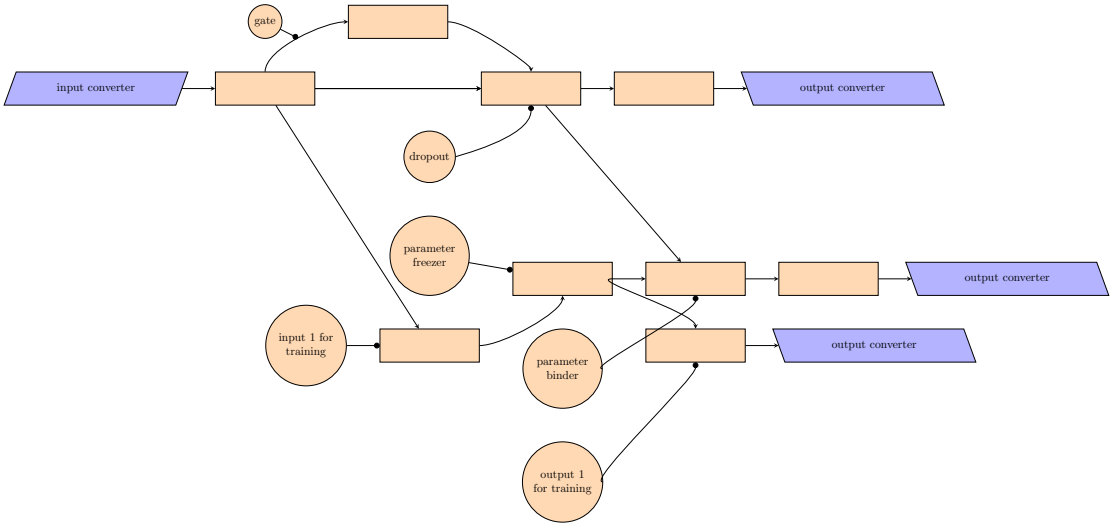


Figure 17: **Example of how to pilot a machine learning pathway**. This diagram represents the architecture of a differentiable neural network, used as the body of a pathway. The controllers are indicated in a circle. The arrow coming out of a controller indicates the spot it pilots. It can be either a layer of neurons or a group of synapses from one layer to the other. We represented here a single IOT controller ("input/output pair for training"), numbered as "input 1 for training", "output 1 for training". Such a controller exploits the differentiability of the DNN to perform a form of supervised learning orchestrated by the deep brain (see the main text).

- a neutral pathway: it implements the identity function, so that its output equals its input
- a generic pathway: it is simply a portion of the deep brain, which is designated as a pathway.

The backbone of a pathway is depicted on Figure 16.

Let us now detail how the body of a pathway can be piloted by the deep brain. We will here restrict to the example of a machine learning pathway, whose body is a deep neural network (DNN). We can see this DNN as a directed acyclic graph (DAG), whose nodes are layers of neurons. To alter the operations performed by a DNN, a possibility is to place controllers at some nodes or edges of that graph. On Figure 17, we give examples of 5 such controllers. Let us describe them, from the simplest to the most complex.

Gate Placed at the level of an edge , it blocks or transmits data conveyed through it. Note that here an edge refers to an edge between layers, so corresponds to a group of synapses.

Dropout unit When activated, it zeroes out the neurons of the layer it controls.

Parameter binder It binds or unbinds the parameters of the neurons of the layer it controls. When parameters are bound, they share the same value. This reduces the number of independent parameters of the network.

Parameter freezer It freezes/unfreezes the parameters of the spot it controls. It it is piloting a node, it acts on the biases of its neurons. If it is piloting an edge, it acts on the weights of its synapses. Frozen parameters cannot change, thus reducing the number of trainable parameters of the network.

IOT pair The most complex controller is an IOT pair (Input/Output pair for Training). It comes as a pair of pilots, one referred as the in-head and the other as the out-head. Each head is placed at a node of the DNN, such that there is a flow of information from the node controlled by the in-head to the node controlled by the out-head. The in-head will impose a state for the layer it controls. Then, the out-head indicates to its controlled layer, what its state should be. The error gap is used to update the trainable parameters along the path from the in-head spot to the out-head spot. Since a DNN is differentiable, gradient descent is used. Thus, an IOT pair allows the deep brain to update some of the parameters of a machine learning pathway, at processing (test) time. The training episode ends when the IOT pair is deactivated by the deep brain.

5.2 Tools for building complex software

We have just described the first component of an artificial brain. Now, we would like to describe its second component: the deep brain. A deep brain can be described as a "neural integrated development environment (IDE) endowed with bounded computational resources". To build such a system, we tried to answer the following question:
How do the IDEs, programming languages and conventions used by human software developers make them so skilled at writing complex algorithms?
Indeed, without the tools offered by an IDE software development would have rapidly hit a ceiling. But instead, it seems that there is no bound on the complexity of the software that humans can build: technological evolution, like biological evolution, seems to be *open-ended*.

This fact matters to us, because an artificial brain is supposed to emulate general intelligence. As a consequence, the deep brain is supposed to be able to realize any mapping between its inputs and outputs, no matter the algorithmic complexity of it. Said otherwise, the deep brain is supposed to build arbitrarily complex software. Before we describe its structure, let us first give some of the core properties of the tools used by human software developers.

- **a hierarchical coordinate system:** it usually takes the form of a file tree, together with row and column indices to locate each character in a file. This structure comes with useful operations: create, delete, copy, paste. These operations are recursive: e.g. when a directory is deleted, its sub-directories are deleted as well. This allows the developer to act on all scales with equal ease.
- **the creation of references, or function definition:** an arbitrary large piece of code can be referred to as a single instruction, which allows the developer to easily build up on their previous work.
- **comments:** they can describe what a piece of code does or how it should be used by other pieces, without the need to execute those pieces
- **errors:** they give information about what went wrong and where, which makes it easier to improve the software. Note that the flow of execution is aborted when an error is detected to avoid undesired consequences on the whole system (like overwriting an important location in your computer's memory)
- **a version control system:** it allows the developer to revert back to a previous working version, develop a new version and commit it only when ready, maintain different versions for different purposes
- **the "print" statement (no black box):** maybe the most powerful debugging tool. It allows a developer to access any intermediate variable.

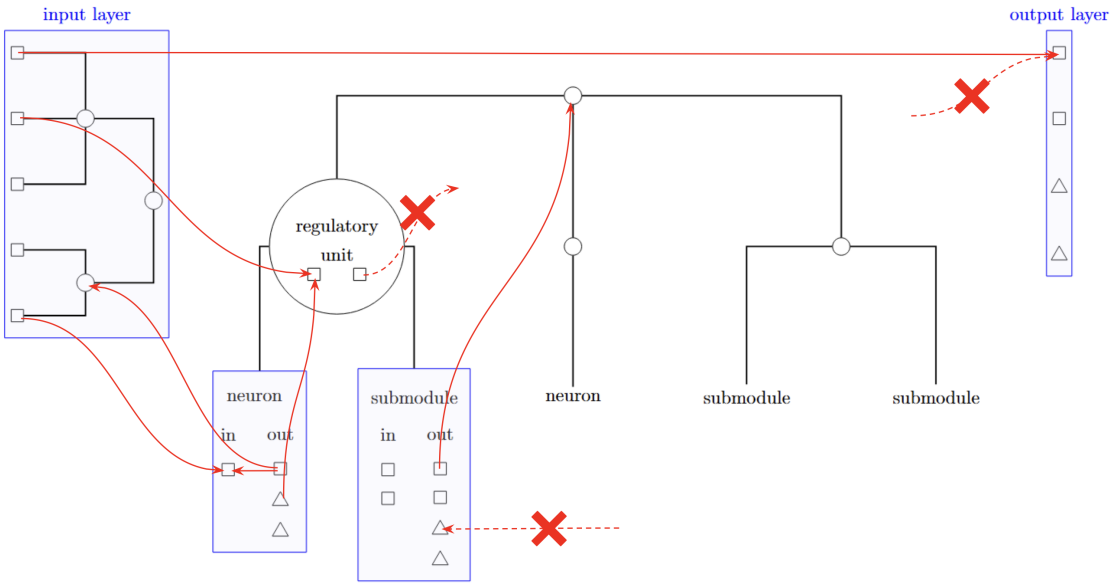


Figure 18: **Structure of a module.** Red arrows stand for synapses. A synapse always comes out of an out-slot and ends in a simple or non-simple slots. The crossed dotted arrows indicate which synapses cannot be formed. Synaptic signals are aggregated only at non-simple slots. Squares (resp. triangles) stand for standard (resp. non-standard) slots and circles stand for regulatory units. The signals received by a module are modulated by its synapse regulatory tree (top left). Then, the module regulatory tree (center) is updated, which imposes how the sub-modules (including neurons) should be updated in turn. Nodes of each tree are updated sequentially, while the sub-modules are updated in parallel.

- **disabling/enabling instructions:** a developer can easily comment out lines in their code, and restore them with equal ease. This is useful when debugging.
- **preservation of a function's scope:** Each call of the same function has its separate scope of local variables. At the end of the function call, these local variables are discarded. This ensures that the same sequence of instructions will be performed, whatever the ordering of the function calls, or even if these calls are parallelized. Note that in PyTorch, this property is partially implemented with the concept of batch dimension: the same module can process different inputs in parallel.
- **branching (the tertiary operator):** The "if" instruction is particular, in the sense that given 2 possible paths, a single one is chosen. Note that in practice, a path is guessed and if it is the wrong one, it is canceled and the other is followed. Two ideas are here: (1) canceling a flow of execution, i.e. reverting some run variables back to a previous state ; (2) disabling some branches (similar to a router).

5.3 The deep brain: practical realization of a NIDE

Let us now detail how we have realized some of those properties in a connectionist setting, to get the concept of a *deep brain*. In the future, we plan to build a differentiable version of it, to enable people to use it with PyTorch. The current version is not, and we have not yet implemented every property listed above.

5.3.1 Modules

At first, a deep brain can be seen as a tree of nested *modules*: we call it the structure tree. The root of this tree is the deep brain itself, while its leaves are neurons. A generic node of the structure tree is called a module. The structure of a module is pictured on Figure 18. It

includes:

- an input layer and an output layer
- a tree of module regulatory units, called the module regulatory tree
- a tree of synapse regulatory units, called the synapse regulatory tree
- a directed graph between slots, called the synaptic network
- a return slot: when activated, the module stops updating its internal state, and returns its output

As long as a module has not returned its output, its input layer is frozen (keeps the same value) and its internal state is updated. There are three causes for a module to return its output:

- it runs out of computational resources
- its internal state has reached a stationary state
- its return slot has been activated

This cause is transmitted to the downstream modules, allowing them to adapt their own computations (see sub-subsection 5.3.2 below). For example, if you know that a module lacked computational resources, then the returned result is prone to errors. Also, you may want to refactor the module, so that it consumes less resources in the future.

Let us now describe the five components of a module.

5.3.2 The input and output layers, and slots

Every component of a module (in/out layers, sub-modules, neurons and regulatory units) has *slots*. These slots allow them to take part in the synaptic network, i.e. to exchange signals with the other components through synapses. A slot can be viewed as either an in-slot (receives synapses) or an out-slot (emits synapses), but never both. The view is chosen depending on our position in the structure tree: a slot from the output layer of a module may send a signal to a slot from the input layer of another, making the latter an in-slot.

Slots hold a value, which must be a float between 0 and 1, and distinguish themselves by 2 aspects:

- **standard or non-standard:** Standard slots are represented in Figure 18 as squares. They update the value they hold upon receiving a synaptic input. Non-standard slots cannot receive a synaptic input. They are represented in Figure 18 as triangles. They update their value depending on their type. For now, we have implemented 5 types of non-standard slots:
  - *error signals:* when such an error is raised, it is forwarded up to the root of the structure tree, and sparks an episode of plasticity (the brain restructures itself, with the knowledge of where the error came from).
  - *functional description:* the purpose of a module is indicated in these non-standard slots (note that generally multiple slots are allocated to the description of a module)
  - *coordinates:* gives the location of the module within the synaptic network of its parent in the structure tree ; the first part of this coordinate indicates the regulatory unit that supervises the module, the second part gives its location with respect to references (anchors) in the synaptic network (anchors are not described here)
  - *metadata:* time of last modification, size (number of parameters), time of last activity
  - *cause for returning:* this slot takes three possible values (0, 0.5, 1) and indicates the status of the internal state of the module when it returns its output (was the return slot active? was a fixed point reached? did it run out of computational resources?)

- **simple or non-simple:** A simple slot can receive at most one synapse. Usually, they are used to forward a value to other slots. For example, the standard in- and out-slots of a module are simple, except if this module is a neuron (see below in the main text). Non-simple slots can receive an arbitrary number of synapses. The value they hold results from the aggregation of the signals received by each synapse. Non-simple slots can differ by their aggregation method (sum, product, maximum, etc.). Contrary to simple slots, they may consume computational resources (see subsubsection 5.3.7 for details about how these resources are managed in a module). A simple interpretation is that the simple slots are crossing points, allowing synapses to project on non-simple slots.

On Figure 18, you can see that the input layer has standard slots only, which are *out* from the point of view of the inside of the module. The output layer has both standard and non-standard slots, but they are all *in*.

When the module is not a neuron, the slots of its input layer can be viewed as the synapses coming in the module, to project on some non-simple slot(s) inside the module, or entering a submodule. Similarly, the slots of its output layer can be seen as the synapses quitting the module to project, after going through the in-slots of other modules, on other non-simple slots.

A neuron differs from the other modules by the fact that it has a single in-slot and a single standard out-slot. Also, its in-slot is both non-simple and standard. Just like the other modules, it has non-standard out-slots. Note that this definition of a neuron is enough to reconcile most models of biological neurons (as long as this model reduces the dendritic tree to a point) with the much simpler neurons used in deep learning.

5.3.3 The synapse regulatory tree

The synaptic signals received by the input layer of a module are modulated by its synapse regulatory tree, before they are (or not) forwarded to the other components. On Figure 18, the synapse regulatory tree is visible on the left. Its non-leaf nodes (circles on the Figure) are *regulatory units*, while its leaves are the slots of the input layer of the module (we will refer to them as the in-slots of the module). Module and synapse regulatory units have the same structure, detailed in Figure 18: only non-simple, in-slots.

A regulatory unit can be in either of $n + 1$ states, where n is the number of its slots. The additional state is the rest state of the unit. When a unit is not resting, i.e. is in an active state, it applies an effect to every leaf below it. In that case, the units that are below the active unit, are skipped and not updated. For this reason, the regulatory units cannot be updated in parallel: the root is updated first, then its children and so on, until either all units have been updated, or every leaf has received some effect.

A regulatory tree is one of the components, that allow the brain to act on itself at various scales with equal ease: applying an effect to a group of incoming synapses is as easy as applying an effect to a group of groups, etc. The state of a regulatory unit is determined as follows:

1. update the value of each in-slot
2. determine the dominant slot (the slot with the largest activation value)
3. if the dominant slot is active enough (typically, > 0), then the unit will enter the state associated to this slot
4. if not, then the unit will enter its rest state

The synapse and module regulatory units differ by their number of slots, and the effects associated to them. Let us describe the effects of a synapse regulatory unit (we plan to extend that list):

- **block:** the signals carried by the affected synapses are blocked, meaning they are not forwarded to any component inside the module
- **synchronize:** the signals carried by the affected synapses are blocked and accumulated in a queue (first in, first out). After some criterion is met (the queues are filled enough),

the blockade is disrupted. This results in all in-slots of the module to fire (i.e. forwarding a signal) together.

- **anti-synchronize:** similar to synchronizing, but each queue is momentarily blocked so that a single in-slot is firing at once

5.3.4 The module regulatory tree

The module regulatory tree is updated after the synapse regulatory tree, and before updating any sub-module. The effect of a module regulatory unit is transmitted recursively to the sub-modules of the modules it is affecting. Since we have already described the general concept of a regulatory tree, we can just give the effects of a module regulatory unit:

- **maintain:** the affected modules maintain their current internal state. The state of a module is the states of all its sub-modules, and the state of a neuron has to be specified (is a tensor generally). Note that what we call "state" here does not include all information about a module (in particular its regulatory trees and synaptic network).
- **set to default:** the affected modules change their state to its default value. The default value of a module consists in the default values of its sub-modules, and the default value of a neuron has to be specified.
- **set to stored:** the affected modules change their state to a stored reference value. At any time step, a module can (not described here) save its current state to a reference value.
- **listen to the synaptic input:** the affected modules update their internal state based on their synaptic input

5.3.5 The synaptic network

A synaptic network is a directed unweighted graph between the slots of a module. The synaptic network must respect the connectivity constraints described in sub-subsection 5.3.2. Note that it is *un*weighted, because the synaptic weights are already stored at the level of the non-simple slots. The other trainable parameters are stored within the neurons. In the future, we will allow a module to modify its synaptic network.

5.3.6 The return slot

The return slot of a module can be any of the standard out-slots inside it. However, it cannot be part of the module input layer. When activated, the module stops updating its internal state. Otherwise, it updates itself by repeating the loop:

1. update the synapse regulatory tree (sequential)
2. update the module regulatory tree (sequential)
3. update the sub-modules (parallel)
4. check whether the output should be returned

5.3.7 The management of computational resources

For now, every module of the structure tree is connected to the same pool of compute power. In the future (for concurrency purpose), multiple pools will be considered. Also in the future, a pool of memory will be implemented. Indeed, we believe that bounded resources both in power (number of operations) and in memory (information stored in the brain) are important regularizer: a bounded compute power favors in/out mappings of low complexity, and so more likely to remain valid for rare inputs (Ockham razor) ; and a bounded memory capacity favors the compression of the information stored in the brain, thus unlocking a path towards abstraction.

In our current implementation, the following optimizations are made:

- if a component (module, neuron, regulatory unit or non-simple slot) has not changed, and receives the same input ; then it is not updated, and does not consume any compute power. Indeed, the output of a module is a deterministic function of its input and its *complete* state (everything inside it).
- modules that maintain their current state (see sub-subsection 5.3.4) are skipped, and do not consume any compute power
- when considering a group of computations that should be applied simultaneously, the cost of the group is computed before they are applied. This allows these computations to be applied all concurrently, or not at all. One example is the linear aggregation of the signals received by an non-simple slot: if n signals are received, the associated computation is $\sum_{i=1}^{n} w_i x_i$. This computation is considered as a single one, with a cost of n multiplications and $n-1$ additions.

6 Prerequisites to AGI (target properties)

Let us enumerate our target properties, and then we will explain them one by one:

- space, time and complexity independence
- self-catalyzed learning
- care for novelty
- self-simplification
- ability to undergo dramatic changes
- modularity bias
- care for generalization
- ability to form a social network

Note that the formalization of these properties is still ongoing work, so some of the following may be highly speculative.

6.1 Space, time and complexity independence

One characteristic of humans, is their ability to draw a link between two pieces of data, no matter how far they are from each other in space or time. Besides, humans are able to learn more and more complex behaviors, with no apparent upper bound: They have access to arbitrarily large levels of algorithmic complexity. This view motivates us to consider three axes, along which some invariance should be observed: space, time and complexity.

6.1.1 Space independence

Generally, for an infinite system with components, space independence means that the number of simultaneously correlated components has an infinite variance but a finite mean. In most cases, the distribution of this number is a power-law of exponent between 2 and 3. In our setting, scale independence can be probed by observing the brain dynamics and the behavior.

In terms of brain dynamics, scale independence means that an arbitrary large number of neurons can federate to participate to the same task. This reflects the ability of the artificial brain to allocate computing power smoothly, without inertia or intrinsic upper bound. This results in several observables that can be sampled from the brain dynamics of each agent:

- number of correlated neurons
- Jaccard diversity of the groups of correlated neurons (is correlation possible in any region of the brain?)
- duration of such correlations (should also be of infinite variance: allocation should be possible for any given duration)

In terms of behavior, space independence means that the behavior of an agent is no more constrained by the spatial structure of the world it lives in. For example, a random walker is completely constrained by the spatial structure of the space it walks in: the time of transition between 2 locations in this space depends on the distance between these locations. On the contrary, when a human being wants to join a distant location, they will favor a particular path once they find it (it may be the shortest path, or the safest, etc.). Also, random walkers will spend as much time in each location, whereas humans have preferred locations: they spent most of their time in a very few locations, and a few of their time in many different locations.

This motivates for the following observables:

- time spent by each agent in each location
- transition time between 2 locations (more precisely, between 2 locations the agent spends a lot of time in)

6.1.2 Time independence

Time independence can also be probed both at the level of brain dynamics and behavior.

In terms of brain dynamics, it is related to memory allocation: an arbitrary amount of information can be retained in memory for an arbitrary amount of time. In the biological setting, this notion of time invariance corresponds to synaptic plasticity. The corresponding observables are related to the synaptic events (modification of a parameter held in memory):

- number of synaptic events at equal time
- duration before a parameter is modified
- integrate events over time and check their distribution across the brain: do events occur in specific areas only? or all over the brain?

In terms of behavior, our notion of time independence means that the behavior of an agent is no more constrained by the time interval separating two events. One consequence is that computing the time autocovariance function of the agent state (brain state, position, action, etc) should make sense no more. Indeed, we expect the agent to be non-stationary (and most likely non-ergodic), as it adapts and innovates indefinitely. Then, its time autocovariance function should be sampled at different time intervals of its trajectory. For a random agent, such functions are expected to be decaying exponentials, that do not depend on the time interval if it is large enough. For the agents we want to evolve, these functions should vary from one time interval to the other, failing to converge. Hence a relevant observable would be the statistical variance of the sampled time autocovariance functions of the following stochastic processes:

- the agent brain state
- the agent position
- the agent action

6.1.3 Complexity independence

Complexity independence can also be probed at both the level of brain dynamics and agent behavior.

Complexity independence is stronger than Turing completeness, because we want the agent to actually execute algorithms of arbitrarily large complexities during its lifetime.

In terms of behavior, this can be probed using various methods. First, the behavior of an agent with complexity independence should not be predictable by a deterministic machine with a finite time horizon. More precisely, we consider the sequence of sensory inputs s_k received by the agent, together with the sequence of actions a_k performed by the agent:

$$\begin{cases} s_k = \text{sensory input received at time tick } k \\ a_k = \text{action performed at time tick } k \\ k \in \mathbb{N} \end{cases}$$

A machine M with time horizon $h \in \mathbb{N}$ is just a map:

$$\begin{aligned} M : S^h \times A^h &\to A \\ ((s_1, a_1), \dots, (s_h, a_h)) &\mapsto M(\vec{s}, \vec{a}) \end{aligned}$$

where S denotes the space of sensory inputs and A the space of possible actions.

Given such a machine, we define its prediction error on a tuple $(\vec{s}, \vec{a}, s_{h+1}, a_{h+1})$:

$$\theta(M; \vec{s}, \vec{a}, s_{h+1}, a_{h+1}) = d(M(\vec{s}, \vec{a}), (s_{h+1}, a_{h+1}))$$

where d is some metric on $S \times A$.

Then, we can sample the distribution of the prediction error from the trajectory of an agent. If this distribution has a finite variance, then we consider the machine M to be a good approximation of the agent. The problem is: is there a finite h for which we can get such a good approximation?

In practice, we use a sequence-to-sequence LongFormer (LongFormer = Transformer with a long attention window), that we fine-tune on trajectories of past agents for different sizes of its attention window (each size corresponds to a different value for h). A LongFormer is possible to use in our setting because of our choice of virtual world (see Section 4), whose states and actions can be turned into a stream of tokens.

Complexity independence may not just be about predictability. To estimate the complexity of the behavior of an agent, we sample redundant patterns in the sequences of tokens and actions written or performed by the agents. In practice, the patterns we mine for are any string of length less than a given size, that occurs at least twice in an agent's trajectory. Then, we can mine for patterns in a hierarchical way, by replacing the original data by a sequence of patterns, and mining for patterns in this sequence (byte-pair encoding strategy). Doing that recursively allows us to probe compositionality, which here consists in an agent concatenating sequences of actions repeatedly, rather than individual actions.

Sampling the number of repetitions, times of occurrences, diversity and sizes of these patterns gives us valuable information about how complex the behavior of an agent is, and how much the agent is able to build up on learned behaviors.

6.2 Self-catalyzed learning

This property grasps the fact that the more knowledge an intelligent agent has, the more likely it will learn new knowledge faster. To detect this property in our setting, we compare the distributions of each target observable sampled at different time intervals in an agent's trajectory. For example, how does the vocabulary of sequences of actions or written tokens (see previous subsection) evolve over the life of an agent? If it slows down or reaches some plateau, then the agent has stopped learning or experimenting new behaviors. Ideally, the vocabulary size of the agent should keep extending at an increasing pace.

Similarly, it should take more and more optimal paths over repetition (see subsection 6.1 about space scale invariance).

6.3 Care for novelty

An intelligent agent is endowed with creativity and curiosity. Thus, an agent should be able to innovate both with respect to itself and to the past agents. Note that in order to innovate with respect to the past agents, transmission of knowledge over multiple generations has to take place (see subsection 6.8 for how to catalyze such a transmission).

To check for this property, an agent should display behaviors that are new to itself, but also to the others. These behaviors are detected by using the method described in subsection 6.1 about complexity scale invariance.

However, novelty can also be probed for by analyzing the brain dynamics. Indeed, although two behaviors may look the same to an external observer, they may hide a very different way of thinking. Take for example the example of a sleeping human and a meditating human. They display the exact same behavior but have very different brain dynamics. Such dynamics correspond to novel ways of thinking, which may turn out useful in the long term. To characterize brain dynamics and compare them with each other, inspiration from neurosciences is more than welcome. Many methods aim at reducing the brain dynamics to a handful of statistical features, but we believe that a more holistic approach such as topological data analysis, would be more appropriate.

6.4 Self-simplification

When confronted with the same task, an intelligent agent should solve it faster and using less internal resources. This is tricky to check in our setting, because we do not know which task

(if any) the agents are trying to solve. However, we can assume that performing the same sequence of behaviors again and again should require less brain resources over time. Then, for each behavioral pattern (see subsection 6.1 about complexity scale invariance), we record how much brain resources were consumed at each occurrence of this pattern.

Ideally, this amount would decrease at each new occurrence of the same pattern. Note that in our setting, how much brain resources were consumed is easy to compute, because computational resources are made explicit and managed by the brain itself.

6.5 Ability to undergo dramatic changes

An intelligent system is always able to step back or to undergo dramatic changes. Dramatic changes can consist in state transitions in the brain dynamics, for example when going from an awake to a sleeping state. Such changes are probed for when an agent receives no sensory input anymore or performs no visible behavior. Changes should also be observed when an agent isolates or on the contrary, joins a group of agents. Dramatic changes in behavior are also possible to observe, using the behavioral patterns extracted according to subsection 6.1. A hint that a change has occurred is that patterns the agent used to display are no more repeated, while the agent displays totally new patterns (e.g. use Kendall similarity between the list of patterns ranked by their frequency of occurrence).

6.6 Modularity bias

The design principle of modularity allows a complex system to be built from independent smaller parts. It is akin to the extension of a language in which an algorithmic solution is searched. Given a programming language, let us consider the problem of finding an algorithmic solution to a given problem, expressed in this language. It may happen that no short solution exists, which would make it difficult to find. However, if we extend the language by combining its primitives, an algorithmic solution may be shorter when expressed in terms of these combinations. This is the idea of modularity: finding building blocks, whose combinations of small sizes can solve a large range of problems. It is equivalent to finding the appropriate programming language, in which a family of given problems can be solved by short algorithms expressed in that language.

Modularity is hard to check directly from generic brain dynamics, because we do not know what algorithms the brain is running, or how it runs them. However, the artificial brains we have designed (see Section 5) can explicitly encapsulate a subgraph of neurons into a single unit, which then behaves just like a neuron. Hence, in the case of the computing systems we have chosen, it is possible to estimate modularity by sampling statistics about these higher-order structures (distribution across the brain, life duration, size, depth (is it a combination of combinations of primitive neurons?), etc).

Modularity can also be probed at the level of the behavior, where it would mean that the agent tends to concatenate known sequences of actions (or tokens) to produce new behaviors. This is already probed for in the mining method described in the part of the subsection 6.1 about complexity scale invariance.

6.7 Care for generalization

An intelligent agent tends to unify its world model through abstraction. It is attracted to generic solutions, that can be applied to a large range of contexts. Ideally, an intelligent agent would apply the exact same algorithm to every context, thus removing the need for domain specific thinking.

This property would be possible to check from the brain dynamics, by computing the contribution of each area of the brain to the observed behavior. Ideally, there would be at least one area that is active in every context. Then, by using information theoretic measures, we can check whether there is a directed flow of information from this area to the output layer. This is required to check whether this area is actually useful to the observed behavior. Note that all areas being active in every context is not a desired property. Indeed, this would go against the

principle of modularity. Ideally, there should be a few areas active very often, while most areas should activate in specific contexts.

6.8 Ability to form a social network

The last property that we consider to be a prerequisite to AGI is the ability of the agents to form a social network. This property is easier to check from the behavior instead of the brain dynamics. Many observables can be defined to investigate the kind of social network the agents are developing. Indeed, interacting socially encompasses the ability to recognize each other, and distinguish one agent from the others, the ability to learn from the others, to communicate with the others, to teach the others and to cooperate with the others.

Cooperation can be probed for by checking whether there are collective behaviors that cannot be reduced to individual behaviors. Said otherwise, there should be individual behavioral patterns that occur *only* within a larger collective behavioral pattern. To investigate this, we need to mine for patterns in the sequence of joint behaviors (see subsection 6.1 for the method of extraction).

Besides those joint patterns, we can build the temporal network of chat co-presence between agents (an edge (i, j, t) is drawn if i and j are connected to the same chat at time t). This network is easy to compute in our setting, due to our design of our virtual world (see Section 4). Building a temporal network is useful because we can use the various tools from the field of social temporal networks to analyze it. In particular, the chat co-presence network can tell us whether some agents prefer to spend time together rather than separated.

However, this network is limited in the sense it does not see wether agents exchange posts with each other. That is why we will also compute the following statistics:

- the size of a post (number of tokens)
- the duration between 2 consecutive posts on the same chat (to estimate the duration of a conversation)
- how many agents participate to a conversation (conversation co-presence temporal network)
- another temporal network, where (i, j, t) is an edge if the agent i posts a message on a chat and the next message posted on this chat is written by the agent j at time t. This network allows us to check whether some agents talk to each other preferentially.

Another meaningful temporal network grasps the spreading dynamics of a behavioral or written pattern: an edge (i, j, t, pat) is drawn if the agent j sees from the agent i an instance of the pattern pat for the first time at time t. We will not detail how we do it in our setting, but this is already implemented. From the spreading dynamics of patterns, we can deduce:

- how many patterns an agent has successfully transmitted to the population?
- how many patterns an agent has learned from the others?

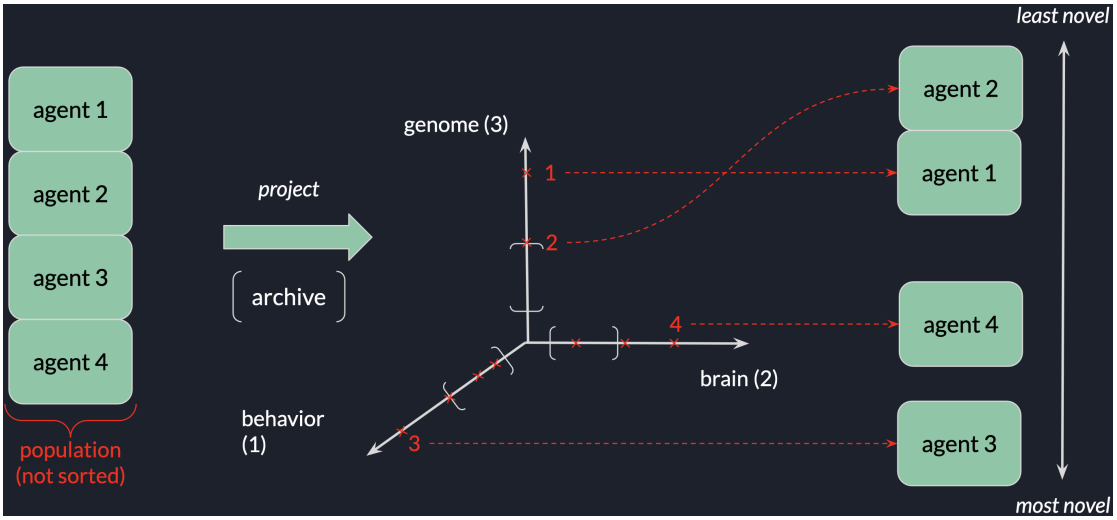


Figure 19: Caption

7 Training policies

The human guides are not supposed to rely fully on their intuition when they modify a population. Before each evolutionary run, they define what we call a *training policy*. In this section, we give two examples of training policies.

7.1 Policy 1: recursive novelty search

In practice, this policy will only be used as a component of another policy. Indeed, it is both very useful and unlikely to be enough for our purpose. It consists in three ordered update objectives:

1. select the agents with the most novel behavior
2. if no novelty in behavior, select the agents with the most novel brain structure and dynamics
3. if no novelty in the brain, select the agents with the most novel genomes (but take care to restrict to non-lethal mutations, if possible, prefer the genomes that have the most numerous sites for non-lethal mutations).

7.2 Policy 2: non-greedy optimization

This policy does not use the same update objective at each cycle of a run. First, agents that are the closest to the target observables will be selected (phase I). This phase goes on until a plateau is reached.

Second, agents with the simplest genomes will be selected, as long as they remain close enough from the target observables (phase II). Note that this phase requires a threshold to be specified. As for the previous phase, it goes on until a plateau is reached.

Third, select the agents according to recursive novelty search (phase III). In this phase, we also restrict to the agents that are close enough to the target observables. This phase cannot end before some minimal genetic diversity has been reached in the population (other threshold required). It keeps going as long as we find some significative improvement on the target observables. Note that although it may not seem intuitive, selecting for novelty can indeed lead to a faster improvement on some goal than the greedy search for that goal. As soon as the agents produced by novelty search are driven away from the target observables, the phase III ends and we are back to the phase I.

The policy 2 is summarized on Figure 20.

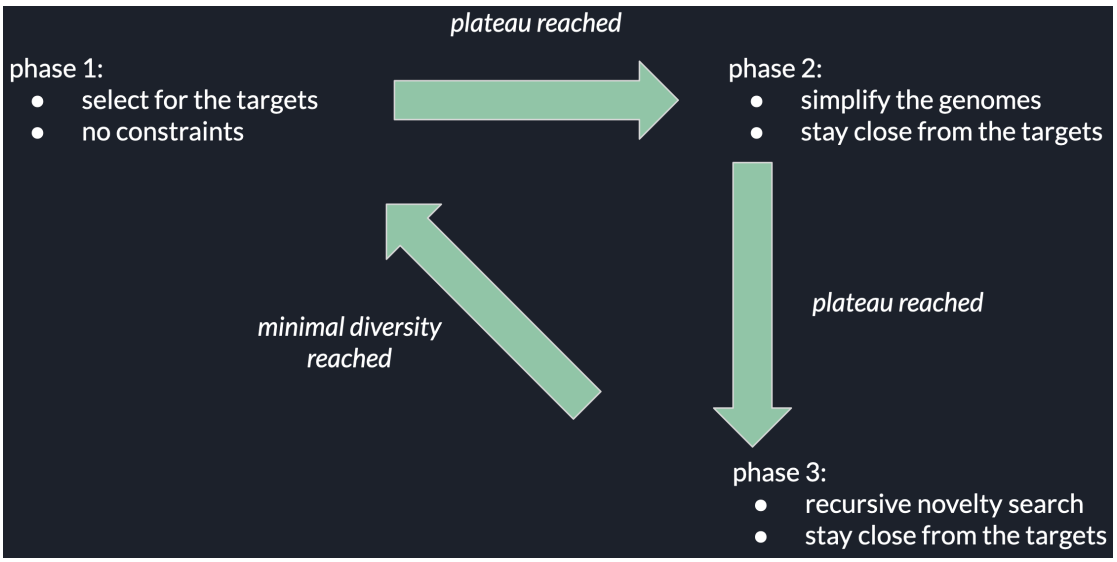


Figure 20: **Caption**. pass